결 과 보 고 서

과제명: A Multicore Extension of the Sparsity-Aware Winograd CNN Accelerator

팀 명: SKKAF

경북대학교 IDEC 2026 창의 회로설계 챌린지

1. 과제 개요

1.1 연구 배경 및 확장 필요성

본 과제는 지정주제에서 구현한 Winograd 연산 구조를 하나의 계산 코어로 삼고, 이를 16개 병렬 코어로 확장한 CNN 가속기를 설계한다. 각 코어는 지정주제의 F(2×2, 3×3) 변환, 16-lane MAC, 행 단위 sparsity-aware 실행 및 ICG를 계승한다. 코어들은 서로 다른 출력 채널을 계산하며, 로컬 SRAM과 multicast를 통해 입력과 중간 결과를 공유한다. 이 구조로 CIFAR-10 VGG16-BN의 convolution과 FC를 연결하였다.

VGG를 처리하려면 연산의 병렬화와 함께 코어 사이의 데이터 공급을 확장해야 한다. 동일한 입력 activation은 여러 출력 채널 계산에 반복 사용되며, 각 layer의 결과는 다음 layer의 입력이 된다. 따라서 계산 코어의 복제뿐 아니라 weight와 변환 tile의 재사용, 분산된 activation의 전달, 저장 영역의 수명을 함께 관리해야 한다.

16개 코어는 출력 채널을 분담하고 각 출력의 입력 채널 reduction을 코어 내부에서 완성한다. 재사용할 변환 weight U와 activation V는 로컬 SRAM에 보관한다. 공통 입력은 multicast로 공급하고, 완료된 출력은 다음 layer에서 직접 읽어 사용한다. 지정주제의 연산 단위를 유지하면서 저장·통신 계층을 확장한 것이 본 설계의 핵심이다.

1.2 설계의 기여

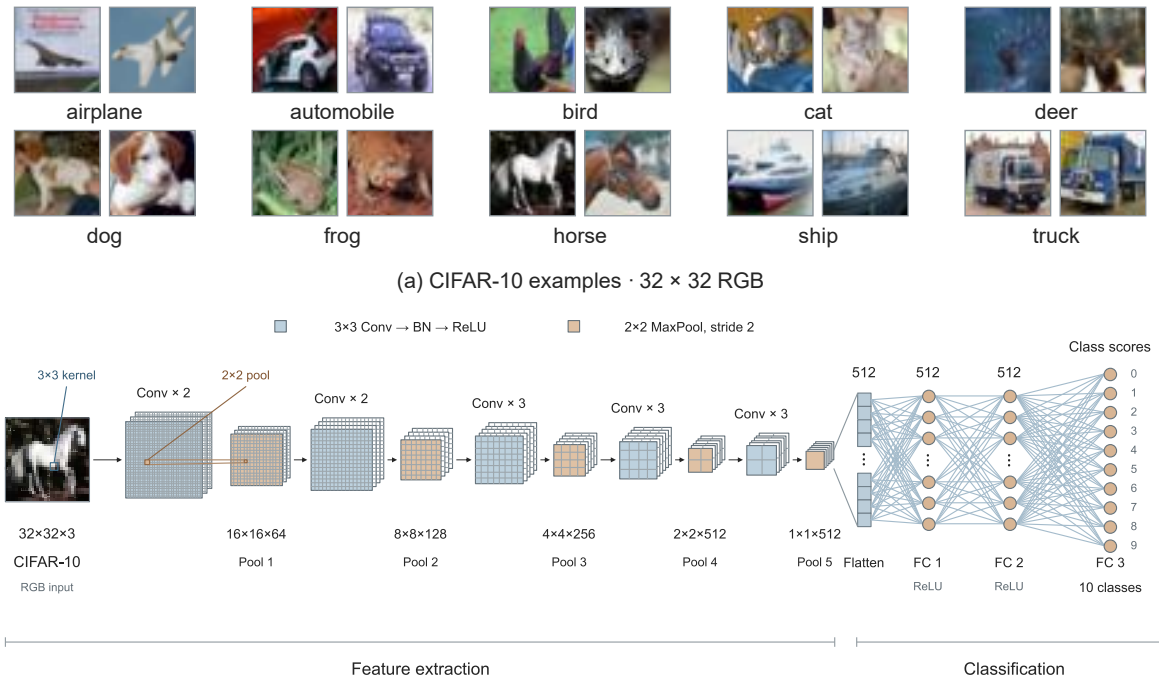
지정주제의 tile 변환·행별 reduction·후처리 흐름을 유지하면서, U/V와 layer 사이 activation을 보관하는 구조를 코어당 96 KiB SRAM으로 확장하였다. 출력 channel 분할로 코어 간 partial-sum 합산을 없애고, 분산 저장된 activation을 multicast로 공급하여 다음 layer의 입력으로 재사용한다. Producer–consumer 제어는 데이터의 공개와 마지막 사용 후 해제를 관리하며, 목적지별 수신 완료 상태는 backpressure에서도 중복 전달을 방지한다.

구현 결과는 정수 모델과 RTL의 출력 일치, SRAM과 multicast의 전달 정확성, 전체 16코어 합성과 타이밍으로 평가하였다. 지정주제의 sparsity-aware 기법이 다른 응용에서도 유효한지 확인하기 위해, 동일 합성 코어의 실행 모드를 전환하여 공급 조건과 희소성에 따른 사이클·에너지 변화를 분석하고 실제 VGG 레이어에 적용하였다.

2. 설계 목표

2.1 대상 모델과 실행 범위

Fig. 1(a)는 10개 클래스로 구성된 CIFAR-10의 32×32 RGB 입력 예시이다 [5]. 대상 모델은 Fig. 1(b)의 VGG16-BN으로, 13개의 3×3 convolution, 5개의 pooling과 FC 512→512→512→10을 수행한다. 반복적인 3×3 convolution은 동일한 Winograd 코어를 layer마다 재사용하기에 적합하다 [1, 2].



(a) CIFAR-10 examples · 32 × 32 RGB

(b) CIFAR-10 VGG16-BN architecture

Fig. 1. CIFAR-10 input examples and target VGG16-BN architecture.

지정주제의 CNN은 28×28 단일 채널 입력, Conv 1→3→3 및 FC 75→10으로 구성된다. VGG에서는 입력 원소가 3.92배인 반면 Conv·FC 가중치는 858개에서 15,239,872개로 약 17,762배 증가한다. 이 차이는 입력 크기보다 깊이와 채널 폭에 따른 연산·저장 요구가 크다는 것을 보여준다.

항목	지정주제 최종 CNN	CIFAR-10 VGG16-BN
입력 H×W×C	28×28×1	32×32×3
Conv / Pool / FC	2 / 2 / 1	13 / 5 / 3
최대 Conv channel	3	512
FC 구조	75→10	512→512→512→10
Conv 가중치 수	108	14,710,464
FC 가중치 수	750	529,408
전체 가중치 수	858	15,239,872
가중치+bias 수	874	15,245,130
Dense Conv+FC MAC / image	28,803	313,725,952

Table 1. Model scale of the designated CNN and CIFAR-10 VGG16-BN.

Table 1의 가중치는 변환 전 Conv kernel과 FC 행렬의 원소 수이며, VGG의 bias는 BN을 결합한 추론 모델 기준이다. MAC 수는 dense 연산식으로 계산하였다. Winograd의 4×4 변환 weight 저장량은 Table 2에 제시한다.

Layer group	Input H×W	Cin → Cout	U8 / core
Conv1 / Conv2	32×32	3→64 / 64→64	0.188 / 4 KiB
Conv3 / Conv4	16×16	64→128 / 128→128	8 / 16 KiB
Conv5 / Conv6–7	8×8	128→256 / 256→256	32 / 64 KiB
Conv8 / Conv9–10	4×4	256→512 / 512→512	128 / 256 KiB
Conv11–13	2×2	512→512	256 KiB
FC1 / FC2 / FC3	1×1	512→512 / 512→512 / 512→10	별도 FC 배치

Table 2. Workload dimensions and local transformed-weight footprint.

Table 2은 16개 코어가 출력 channel을 균등 분담할 때의 로컬 U8 payload를 나타낸다. 4×4 변환 원소를 각각 signed 8-bit로 저장하므로 코어당 payload는 16·Cin·Klocal byte이며, 홀수 Cin의 word 정렬 padding은 제외한 값이다. 코어당 U SRAM은 32 KiB로 Conv5까지 전체 로컬 U를 저장할 수 있다. 이후 layer에서는 출력 channel 묶음별로 U를 교체하고 해당 묶음의 계산 동안 재사용한다.

2.2 지정주제와의 설계 연결

항목	지정주제	자유주제
연산 단위	F(2×2, 3×3), 16-lane MAC	동일 단위의 16개 코어
생략 방식	영행 압축 및 zero-job 우회	행별 mask와 순서 보존 유지
V 보관	소규모 buffer와 channel 선택	SRAM CB에 저장, 마지막 재사용 후 해제
중간 activation	소형 공유 저장소	코어별 A/B SRAM, 레이어 간 역할 전환
다음 레이어 공급	단일 코어 내부 경로	shard 주소 변환 및 multicast
정수 형식	W4, 내부 V INT11	U8/V8 저장, 별도 scale 복원
실행 제어	고정 layer 순서	host command 기반 Conv/FC 실행

Table 3. Preserved computation and extended data management.

Tile 생성과 transform, input-channel reduction의 연산 원리는 지정주제에서 유지하였다. 자유주제에서는 SRAM의 동기 읽기 지연과 단일 포트 경합을 반영하고, layer command로 받은 텐서 크기와 주소 범위를 순회하도록 저장·제어 구조를 확장하였다. 고정된 연산 단위를 channel과 layer에 걸쳐 재사용하는 것이 두 설계를 연결하는 원칙이다.

하드웨어 구성은 16코어×16 MAC lane, 코어당 12개 8 KiB SRAM이다. 외부 데이터는 64-bit ready/valid 인터페이스로 전송하며, host가 텐서 크기와 주소를 포함한 layer command를 발행한다. Conv와 FC는 같은 저장·전달 경로를 사용한다.

3. 가속기 아키텍처 설계

3.1 16코어 구성과 데이터 흐름

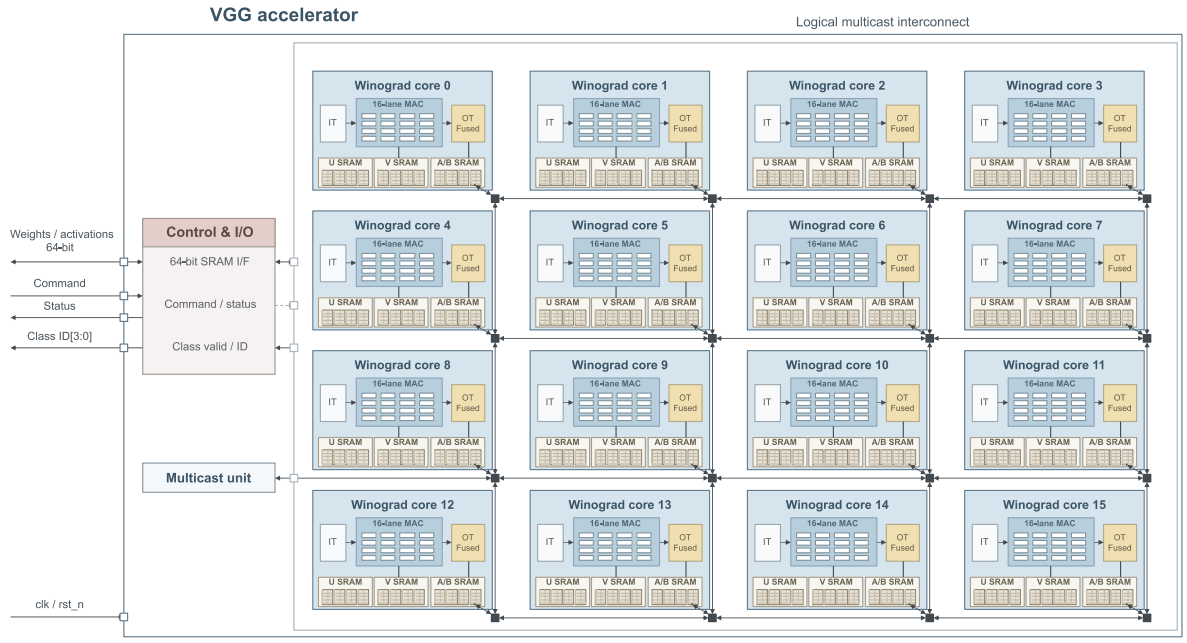


Fig. 2. Sixteen-core Winograd accelerator and shared activation supply.

최상위 회로는 host 접근과 layer command, 공통 activation 공급기, multicast fanout 및 16개 계산 코어로 구성된다. 각 코어는 로컬 U/V와 activation SRAM을 사용하며, 자신에게 할당된 출력 channel의 weight로 계산한다. 명령은 텐서 크기, 입력·출력 주소, A/B phase, pooling, sparse 실행 및 FC 모드를 지정한다. NVDLA의 layer 단위 구성·실행 방식과 같이 [3], host가 layer 실행과 완료를 관리하고 실제 연산과 데이터 전달은 RTL 내부에서 수행한다.

입력 activation은 source SRAM에 저장한 뒤 공통 공급기가 tile 순서로 읽어 multicast한다. 각 코어는 수신 데이터를 4×4 tile로 조립하여 input transform에 넣고, 변환 결과를 V SRAM에 저장한다. 모든 코어가 사용하는 공통 입력을 한 번 적재하여 칩 내부에서 분배한다.

다음 레이어에서 입력 채널은 이전 레이어의 여러 코어에 나뉜 출력으로부터 구성된다. 공급기는 입력의 논리 채널과 공간 좌표를 source core 및 SRAM 주소로 변환한다. 이때 입력 텐서 전체를 한 코어에 복사하여 합친 뒤 계산할 필요는 없다. 필요한 순서로 각 shard를 읽어 모든 소비 코어에 공급하면 각 코어가 자신의 출력 채널에 대한 reduction을 완성할 수 있다.

3.2 출력 채널 분할과 레이어 연결

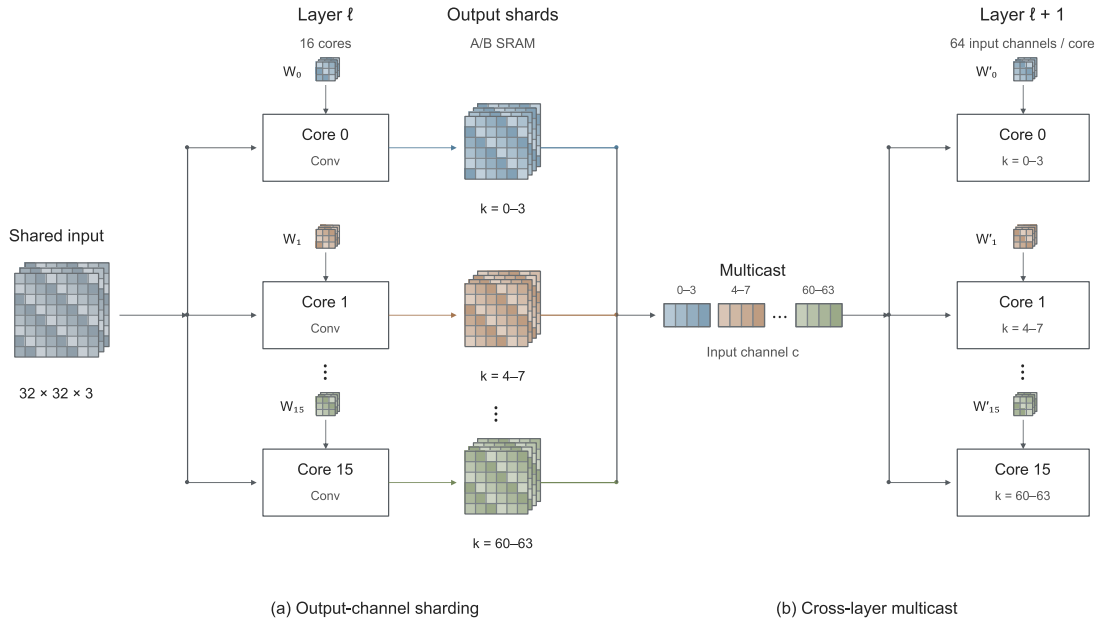


Fig. 3. Output-channel sharding and activation reuse across layers.

$$M_k = \sum_c \left(U_{k,c} \odot V_c\right), \quad Y_k = A^T M_k A$$

한 코어가 담당하는 출력 채널 k에 대해서는 모든 input channel c를 누산한다. 따라서 같은 출력의 partial sum을 여러 코어가 나누어 보유하지 않으며 코어 간 reduction tree가 필요하지 않다. 각 코어가 생성한 출력은 서로 다른 채널의 완성값이다. 다음 레이어에서는 이 채널들을 입력으로 읽어 다시 로컬 reduction을 수행한다. 코어 간 통신 대상은 partial sum이 아니라 완성된 activation이다.

출력 channel 분할에서 V는 공통 입력으로부터 생성되고, U는 코어가 담당하는 출력 channel에 따라 달라진다. 입력 activation은 multicast로 공유하며 U는 각 코어에 적재한다. 로컬 SRAM에 들어가는 U는 상주시키고, 용량을 초과하면 출력 channel 묶음별로 교체한다.

Activation 저장소의 A/B는 입력 전용·출력 전용으로 고정하지 않는다. 한 layer에서 A를 읽고 B에 결과를 기록한 뒤, 완료된 B를 다음 layer의 입력으로 지정하고 A를 출력 영역으로 사용한다. 명령의 phase와 base address가 역할을 정한다. 자기 코어가 생성한 결과도 동일한 source 주소 규칙으로 읽으므로 역할 전환을 위해 반드시 B에서 A로 복사할 필요는 없다.

3.3 SRAM 배치와 producer–consumer 제어

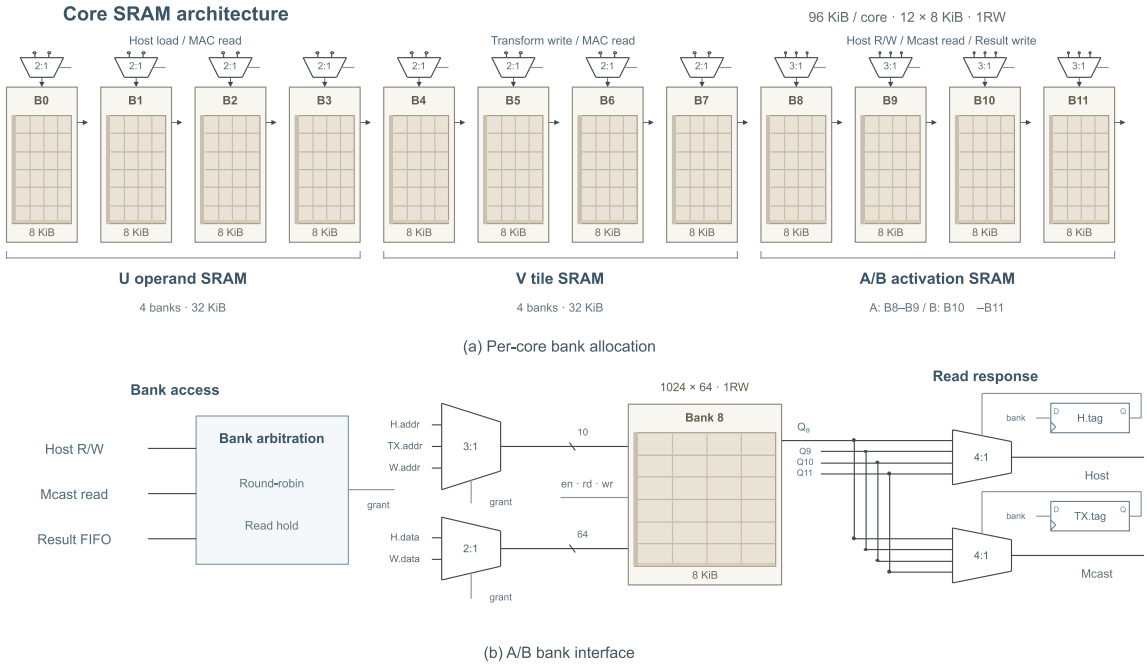


Fig. 4. Per-core SRAM allocation and A/B bank arbitration.

영역	Bank 수	용량	접근 및 수명
U	4	32 KiB	MAC 행별 weight, 상주 또는 묶음 교체
V	4	32 KiB	변환 행 저장 및 일부 line history 주소
Activation A	2	16 KiB	phase에 따른 입력 또는 출력
Activation B	2	16 KiB	phase에 따른 출력 또는 입력
총계	12	96 KiB	16코어 총 192개, 1.5 MiB

Table 4. Physical SRAM allocation per core.

각 macro는 1024×64-bit 동기식 1RW SRAM이다. 서로 다른 bank는 병렬 접근할 수 있으나 같은 bank의 읽기와 쓰기는 중재해야 한다. A/B 각각의 물리 bank 배치는 접근 포트를 구성하고, layer별 A/B 역할 전환은 입력과 출력 데이터의 수명을 관리한다. 주소·read/write 요청의 실행은 bank 인터페이스가 담당하며 마지막 사용과 영역 반환은 상위 제어가 판단한다.

Circular buffer(CB)는 재사용할 데이터의 저장 수명을 관리한다. Producer는 빈 slot을 예약하여 데이터를 기록한 뒤 읽기 가능한 범위를 공개한다. Consumer는 공개된 데이터만 읽고, 같은 V tile을 참조하는 마지막 출력 channel의 계산이 끝났을 때 slot을 해제한다. 이후 producer는 반환된 공간을 새 tile에 사용할 수 있으며, 용량이 허용하는 동안 생산과 소비를 중첩한다.

V의 생산 중에도 이미 기록된 channel은 watermark를 통해 소비할 수 있다. 따라서 모든 channel의 변환이 끝날 때까지 일괄 대기하는 장벽을 두지 않는다. 단, U8/V8 밀집 저장에서는 두 channel의 한 행을 한 SRAM word에 묶으므로, word가 완성되어 네 행의 쓰기가 확인된 범위까지 공개한다. 데이터보다 mask가 먼저 유효해지거나 일부 행만 갱신된 tile을 읽지 않도록 공개 시점을 정하였다.

Activation SRAM은 host 접근, source read, result write를 bank별 round-robin 방식으로 중재한다. Read 응답은 소비자가 수락할 때까지 유지한다. V SRAM은 MAC read와 transform write를 중재한다. 물리 포트의 요청 승인·응답 보존은 bank 제어가, U의 상주와 V의 마지막 사용 후 해제는 상위 제어가 담당한다.

3.3.1 공개 SRAM 매크로의 사양과 RTL 연결

각 SRAM bank는 The OpenROAD Project의 공개 ASAP7 SRAM 모델 srambank_256x4x64_6t122를 사용하였다 [4]. 기능 검증에는 behavioral Verilog를, 합성에는 Liberty 기반 매크로 인스턴스를 사용하였다. 저장 배열을 FF로 펼치지 않고 8KiB 단위의 메모리 경계로 유지하며, bank별 중재·CB 수명 관리·multicast 전달 제어는 본 설계에서 구현하였다.

항목	사용 모델 사양
용량 / 구성	1024 words × 64bit = 8KiB / single-port
주소 / 데이터 폭	10bit / 64bit
동작	상승 에지 동기식 읽기·쓰기; 동시 접근은 중재
출력 / 부가 기능	다음 읽기까지 출력 유지; byte-write mask·reset 없음
LEF 치수 / 면적	30.348 × 77.76μm / 2,359.86μm²
공개 모델 / 라이선스	ASAP7 SRAM 6t122 계열 / BSD-3-Clause [4]

Table 5. SRAM model specifications used by the accelerator.



```
// 1024 x 64-bit synchronous SRAM
srambank_256x4x64_6t122 u_sram (
    .clk     (clk),
    .ADDRESS (addr),
    .wd      (wdata),
    .banksel (en),
    .read    (rd_en),
    .write   (wr_en),
    .dataout (rdata)
);
```

Fig. 5. (a) Public ASAP7 SRAM layout example; (b) SRAM instance in the accelerator wrapper.

(a)는 공개 srambank_64b.gds의 레이아웃 예시이며, (b)는 실제 SRAM wrapper의 연결 코드이다. 예시 GDS와 사용한 8 KiB 모델의 물리 뷰 대응은 미확인이다. Table 5의 치수와 면적은 사용 모델의 LEF에서 추출하였다.

3.4 변환 데이터의 밀집 저장과 weight 상주

U와 V의 비트폭을 줄이더라도 64-bit word에 빈 공간을 남기면 저장량과 전송량은 감소하지 않는다. U8은 한 행의 네 원소를 32 bit로 묶고 인접한 두 input channel을 한 word에 저장한다. 네 행은 각각 다른 U bank에 배치한다. 로컬 출력 채널 k와 입력 채널 c의 word 주소는 다음과 같다.

$$W_{out} = \left\lceil \frac{C_{in}}{2} \right\rceil, \quad addr_U(k,c) = base_U + k \cdot W_{out} + \left\lfloor \frac{c}{2} \right\rfloor$$

전체 로컬 U의 상주 조건은 Klocal·Wout ≤ 1024이다. 상주 U는 공간 tile과 출력 window가 바뀌어도 보존하고 주소 포인터만 갱신한다. 용량을 초과하는 layer는 현재 출력 묶음의 U를 적재·재사용한 뒤 다음 묶음으로 교체한다.

V8은 네 row bank 각각에 두 channel의 행을 묶어 저장한다. 첫 행을 32-bit register에 보관한 뒤 다음 channel의 행과 합쳐 기록하므로 SRAM read-modify-write가 필요하지 않다. 홀수 마지막 channel은 padding으로 정렬한다. Cin=512인 한 공간 tile의 V payload는 V16의 16 KiB에서 V8의 8 KiB로 감소한다.

V와 mask는 동일한 논리 tile/channel 수명을 갖는다. Weight를 바꾸어 같은 V를 다시 사용할 때 mask를 재계산하거나 먼저 해제하지 않는다. MAC의 네 행은 서로 다른 유효 channel을 선택할 수 있고, SRAM 응답에 각 행의 half-word 선택 정보를 붙여 해당 U/V를 일치시킨다. 포화 누산의 결과를 유지하도록 각 행 내부의 channel 순서는 보존한다.

3.5 Multicast와 전달 완료 제어

Multicast fanout은 payload와 목적지 mask를 받아 선택된 코어에 전달한다. 목적지별 delivered 상태를 유지하여 이미 수락한 코어에는 같은 word를 다시 보내지 않는다. 일부 코어가 정지하면 해당 수신만 기다리고, 모든 지정 목적지가 수락한 뒤 entry를 반환한다. Upstream ready는 FIFO의 full 상태로 결정하여 수신측 ready가 송신측까지 긴 조합 경로로 이어지지 않도록 하였다. 데이터의 저장 수명은 CB와 layer 제어가, 목적지별 전달 완료는 fanout이 담당한다.

3.6 FC 실행과 layer 경계

FC에서는 transform을 거치지 않고 activation vector와 weight를 MAC에 공급한다. FC1과 FC2의 512개 출력은 16개 코어가 32개씩 담당한다. 다음 FC는 분산된 이전 출력을 vector 순서로 읽어 각 출력의 reduction을 수행한다. 마지막 FC는 한 코어가 10개 출력을 계산하고 argmax로 분류 결과를 생성한다.

각 layer의 결과는 SRAM 기록까지 완료된 뒤 다음 layer의 입력으로 공개한다. 이를 위해 연산기 done뿐 아니라 출력 큐의 drain을 확인한 뒤 output region을 commit한다. 연산이 끝났어도 SRAM 쓰기가 남아 있으면 layer 완료를 지연하여 다음 layer가 이전 데이터를 읽는 것을 방지한다.

4. 검증 및 구현 결과

검증 항목은 정수 모델과 RTL의 출력 일치, 코어 간 데이터 전달, 연산량·사이클·에너지 절감 및 합성 면적·타이밍이다. 전력 비교에는 최종 넷리스트의 계산 코어와 내부 U/V SRAM 8개를 사용하였다. ICG를 유지한 상태에서 sparsity-aware 실행의 OFF/ON을 비교하고, 1,430 ps 클록의 게이트 활동 파형을 OpenROAD에 반영하였다.

4.1 정수 모델과 기능 검증

학습된 VGG16-BN의 batch normalization을 Conv weight와 bias에 결합하고, 변환 operand의 저장 정밀도를 평가하였다. V는 input transform 이후 값의 범위를 기준으로 layer별 shift를 calibration하였다. Table 6은 CIFAR-10 test 10,000장의 software 추론 정확도이다.

변환 operand 저장	정답 / 10,000	정확도	U16/V16 대비
U16 / V16	9,411	94.11%	기준
U8 / V16	9,415	94.15%	+0.04%p
U8 / V8	9,394	93.94%	−0.17%p

Table 6. Software accuracy of transformed-operand quantization.

U8/V8의 정확도는 93.94%로 U16/V16 대비 0.17%p 낮았으며, 두 구성의 예측은 9,898장에서 일치하였다. 변환 operand의 저장량을 절반으로 줄이면서 분류 정확도를 유지하였다.

$$q_V = clamp\left(\left\lfloor \frac{V + 2^{s-1}}{2^s} \right\rfloor, -127, 127\right)$$

s=0에서는 rounding bias를 더하지 않으며, signed round-half-up과 대칭 범위 −127…127을 사용한다. Calibration 값은 Conv1에서 s=2, 나머지 convolution에서 s=1이다. 전체 V16 transform 이후 qV와 zero-row mask를 함께 등록하고 MAC 입력에서 qV·2^s로 scale을 복원한다. 이에 따라 V8 정수화로 새로 생긴 영행도 실제 소비 operand와 일치한다. U8/V8은 SRAM 저장 형식이며 MAC의 기존 내부 연산 폭은 유지한다.

U8/V8 Conv→FC1→FC2→FC3 연결 시험은 중간 activation의 재적재 없이 출력 202 words와 class=8을 확인하였다. 이어서 reset 없는 Conv 복귀, 잘못된 명령 길이 거절 및 영역 반환을 검증하였다. 원본 검증 로그는 Fig. 10에 제시한다.

전체 모델의 선행 검증은 U8/V16 구성으로 CIFAR-10 test image 0 한 장을 실행하였다. 13 Conv, 5 pool, 3 FC 및 argmax에서 중간 출력 22,986개의 64-bit word가 정수 기준 모델과 일치하고 최종 class=3(cat)이 label 및 software 예측과 같았다. 최초 3,072-byte image 이후의 중간 activation은 host 재적재 없이 SRAM과 내부 공급 경로로 전달하였다. 이 결과는 전체 분류 경로의 연결성을 검증한 것으로, 최신 U8/V8 기준본의 연속 전체 모델 검증 및 latency와는 구분한다.

4.2 Multicast와 activation 공급 검증

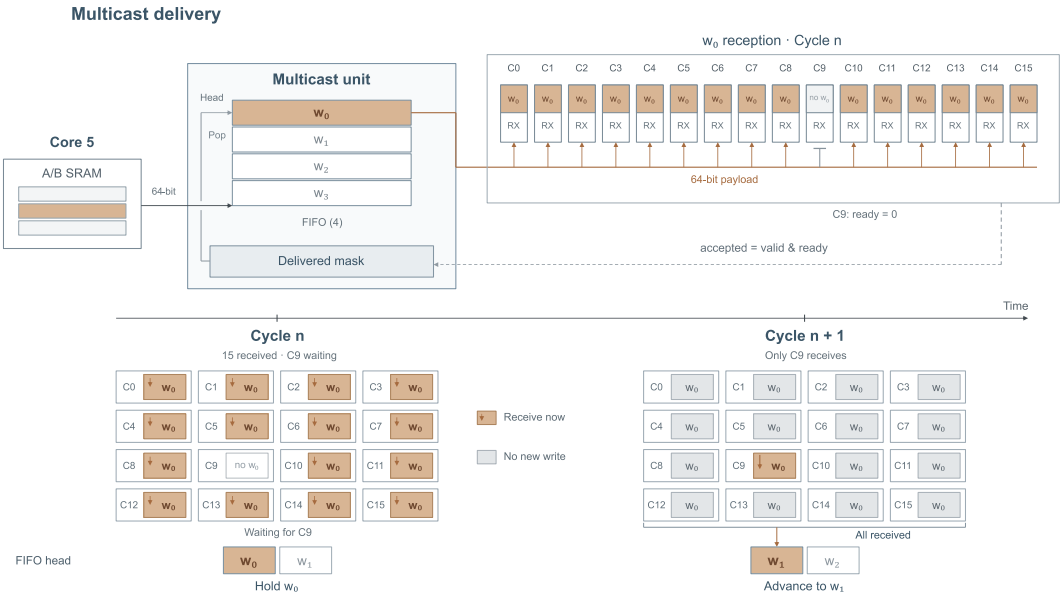


Fig. 6. Multicast delivery with destination-wise acceptance tracking.

Fanout 단위검증에서는 16개 목적지의 독립 backpressure와 부분·빈 mask를 적용하였다. 192개 입력에 대한 목적지별 총 1,043회 전달을 확인하였으며, 이미 수락한 수신자의 상태와 전송 순서가 유지되었다. 연속 공급·수신 조건에서는 192 words가 시작·종료를 포함하여 195 cycles에 완료되었다. FIFO가 full인 cycle에 pop이 발생하면 새 입력은 다음 cycle부터 수락한다.

동일 payload를 코어마다 외부에서 16회 보내는 방식과 한 번 적재 후 내부 multicast하는 방식을 비교하였다. 공급 subsystem 실험은 첫 host write 제공부터 실제 SRAM read, tile 순서 공급, 마지막 수신 FIFO의 drain까지 포함한다. 반복 적재 대조군은 각 코어의 적재가 끝나면 읽기를 시작하여 뒤 코어의 적재와 중첩하도록 구성하였다. 따라서 단순히 16회 적재 시간을 직렬 합산한 대조군은 아니다.

입력 NHWC	외부 공급 간격	반복 적재 (cycles)	단일 적재+MC (cycles)	가속비
[1,4,4,32]	1 cycle/word	6,531	5,572	1.17×
[1,4,4,32]	4 cycles/word	9,600	5,761	1.67×
[1,8,8,64]	1 cycle/word	62,211	54,532	1.14×
[1,8,8,64]	4 cycles/word	86,784	56,065	1.55×
[1,8,8,256]	1 cycle/word	248,835	218,116	1.14×
[1,8,8,256]	4 cycles/word	347,136	224,257	1.55×

Table 7. End-to-end activation supply through SRAM and multicast.

입력 크기가 D byte일 때 외부 activation 전송량은 16D에서 D로 93.75% 감소하였다. SRAM 읽기와 tile 순서 변환을 포함한 전체 공급 시간은 1.14–1.67배 단축되었다. 수신측을 항상 ready로 둔 공급 경로 시험이며, 모든 수신 word를 독립 주소 oracle와 대조하였다.

4.3 공급 조건과 희소성에 따른 사이클·에너지 절감

Sparse OFF와 ON은 모두 동일한 Winograd 연산을 수행한다. 본 절은 여기에 추가한 행별 압축과 zero 작업 우회의 효과를 평가한다. 한 출력 channel의 reduction에서 행 r의 유효 input channel 수를 nr라 하면, 모든 데이터가 준비된 경우 행별 작업을 병렬로 발행하는 데 필요한 최소 beat 수는 max(n0,n1,n2,n3)이다. 한 beat는 동일 MAC 배열에 대한 한 번의 입력 수락이며, 활성 lane 투입량은 4Σnr이다.

$$N_{beat} \geq max(n_0, n_1, n_2, n_3), \quad N_{lane} = 4\sum_r n_r$$

활성 lane 투입량은 부분적으로 비활성인 beat까지 반영한다. 반면 실행 사이클은 행별 최대 작업량과 공급·제어 대기에 의해 결정된다. 이를 분리하여 관찰하기 위해 텐서 형상과 가중치를 고정하고 변환 후 zero-row 비율과 입력 공급 간격을 조절하였다. 에너지는 각 명령의 시작부터 완료까지 측정한 평균 전력과 실행 시간의 곱으로 계산하였다.

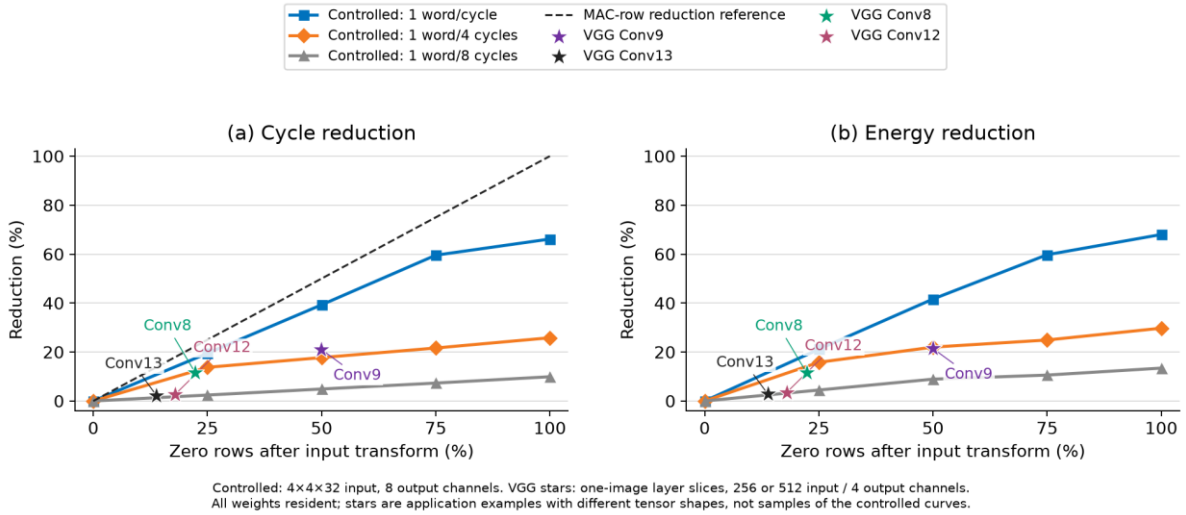


Fig. 7. Cycle and energy reduction under controlled supply conditions, with actual VGG layer examples.

통제 실험의 입력은 [1,4,4,32], 로컬 출력은 8채널이며 U8 가중치 4 KiB를 SRAM에 상주시켰다. 64-bit 입력 word의 제공 간격을 1·4·8 cycles로 설정하고, 공간 activation의 채널별 타일을 주기적으로 0으로 구성하여 변환 후 zero-row 비율을 0·25·50·75·100%로 조절하였다. OFF/ON 총 30조건에서 출력과 MAC 행 투입 횟수가 독립 정수 기준값과 일치하였다. 초기 U 적재를 제외하되 실행 중 SRAM 접근과 제어 대기는 포함하였다. 100%는 모든 입력 타일이 0인 경계 조건이다.

Zero-row 비율 50%에서 공급 간격 1 cycle/word는 사이클 39.28%, 에너지 41.64%를 줄였다. 간격이 4 cycles/word이면 각각 17.73%, 21.99%, 8 cycles/word이면 4.89%, 8.92% 감소하였다. 동일한 연산 생략량에서도 공급 제한에 따라 실현되는 시간·에너지 이득이 달라진다. 점선은 zero-row 비율과 일치하는 MAC 행 투입 감소의 기준이며 사이클·에너지의 보장 상한은 아니다.

별표는 CIFAR-10 test image 0의 실제 VGG activation과 가중치를 사용한 출력 4채널 슬라이스이다. Conv8: 영행 22.31%, 사이클 11.57%, 에너지 11.52% 감소; Conv9: 영행 49.93%, 사이클 21.08%, 에너지 21.51% 감소; Conv12: 영행 17.92%, 사이클 2.69%, 에너지 3.42% 감소; Conv13: 영행 13.82%, 사이클 2.24%, 에너지 2.81% 감소. 실제 사례는 입력 형상과 행별 zero 배치가 통제 패턴과 다르므로 곡선 위에 놓일 필요는 없다. 통제 곡선은 해당 패턴의 반응을, 별표는 실제 적용 결과를 나타내며 데이터셋 평균 절감률을 뜻하지 않는다.

4.4 Winograd와 sparsity에 따른 레이어별 연산량

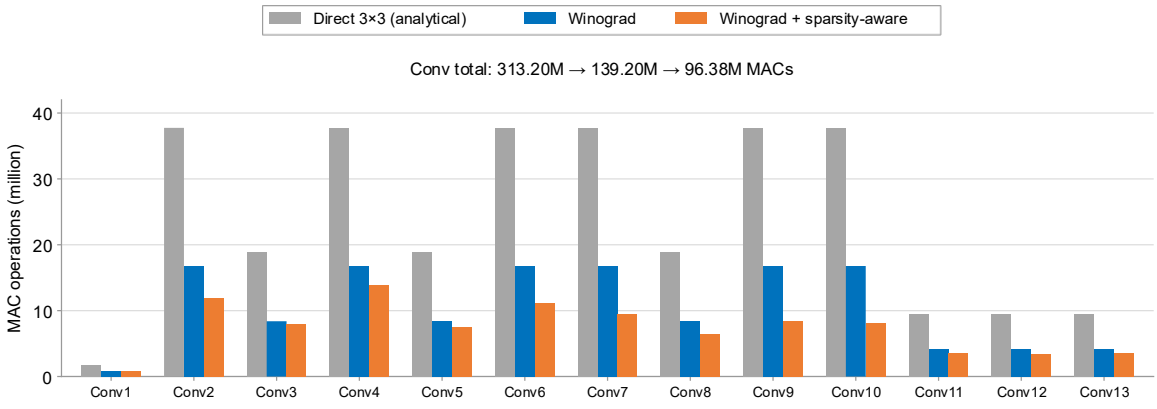


Fig. 8. Layer-wise MAC counts for direct convolution, Winograd, and sparsity-aware Winograd.

Fig. 8은 동일한 VGG convolution의 direct 3×3, Winograd, sparsity-aware Winograd MAC 횟수를 비교한다. Direct는 Hout·Wout·Cin·Cout·9, Winograd는 (Hout/2)·(Wout/2)·Cin·Cout·16으로 계산한다. 비교 범위는 convolution의 곱셈·누산이며 변환의 덧셈·뺄셈, 후처리 및 FC는 제외한다.

Winograd와 sparsity-aware Winograd는 CIFAR-10 test image 0의 activation으로 13개 convolution을 16코어에서 각각 실행한 RTL 계측값이다. 동일 input/weight의 sparse OFF/ON 출력을 정수 모델과 대조하였다. MAC 입력 수락 시 활성 lane 수를 누적했으며, sparse OFF의 계측값은 모든 layer에서 Winograd 산출량과 일치하였다.

13개 convolution의 MAC 횟수는 direct 313,196,544회, Winograd 139,198,464회, sparsity-aware Winograd 96,382,208회이다. Winograd가 direct 대비 55.56%를 줄이고, sparsity-aware 실행이 남은 연산의 30.76%를 추가 생략하여 총 69.23% 감소하였다. 한 이미지의 layer별 추가 생략률은 Conv1의 0.20%부터 Conv10의 51.83%까지 분포하였다.

Fig. 8의 전체 16코어 공급 경로를 포함한 레이어 시험에서는 MAC 투입량 감소에도 불구하고 실행 사이클 감소가 0–0.124%에 머물렀다. Fig. 7의 계산 코어 평가와 함께 보면, 연산 생략의 효과를 활용하려면 연산기뿐 아니라 activation 공급과 데이터 재사용이 함께 뒷받침되어야 한다.

4.5 전체 합성과 SRAM 경계 타이밍

16코어 전체 회로를 합성한 결과, 계산 코어 16개와 SRAM macro 192개를 포함한 netlist를 얻었다. SRAM은 제공된 Liberty 모델로 매핑하였다. 구조 검사에서 오류는 없었으며, 최종 면적과 타이밍은 Table 8에 정리하였다.

항목	최종 구현
최소 주기	1,377.62 ps
STA 환산 주파수	725.89 MHz
Liberty 면적, SRAM 포함	742,748.20 μm²
FF / ICG	199,058 / 160
계산 코어 / SRAM macro	16 / 192

Table 8. Final full-cluster synthesis and pre-layout timing results.

Table 9는 sparsity 제어의 추가 면적과 ICG 셀의 면적 비중을 정리한다. Sparsity 비교본은 sparse_enable을 0으로 고정하고 상수 전파와 사용되지 않는 논리만 제거하였으며, 나머지 셀 매핑과 ICG 10개를 유지하였다. 비교본과 제안 코어의 면적은 U/V SRAM 8개를 포함하여 각각 35,752.35 μm²와 35,958.29 μm²이다.

Table 9. Sparsity-control area overhead and ICG-cell area contribution.

측정 항목	면적 (μm²)	비율
Sparsity 추가 면적 · SRAM 포함	205.94250	0.576%
Sparsity 추가 면적 · SRAM 제외	205.94250	1.221%
ICG 셀 합계 · 코어당 10개	2.62440	0.0073%
ICG 셀 합계 · 전체 클러스터 160개	41.99040	0.0057%

Sparsity 비율은 기능을 끈 비교본 면적 대비 증가율이며, ICG 비율은 해당 최종 회로 면적 중 ICG 셀이 차지하는 비중이다. ICG 셀 비중에는 enable 생성 논리와 클럭 배선이 포함되지 않으며, ICG 제거 전후 전체 면적의 차이를 뜻하지 않는다.

합성 후 STA의 최소 주기는 1,377.62 ps, 환산 주파수는 725.89 MHz이다. SRAM을 포함한 Liberty 셀 면적은 742,748.20 μm²이다. Fig. 9는 합성·타이밍 리포트의 해당 항목을 발췌한 것이다.

```
[1_Post_synthesis.rpt]
Post synthesis report_clock_min_period
-------------------------------------------------------------------------
clk period_min = 1377.62 fmax = 725.89

[synth_stat.txt]
   Chip area for top module '\vgg_sixteen_core_cluster': 742748.196420
     of which used for sequential elements: 63347.563080 (8.53%)

[synth_check.txt]
Found and reported 0 problems.
```

Fig. 9. Excerpts from synthesis, timing and structural-check reports.

```
[U8/V8: integration.log]
PASS FC_HEAD Conv_FC1_FC2_FC3 checked_words=202 class=8 initial_activation_words=4
  no_intermediate_TB_reload
PASS FC_MODE_RETURN rejected_bad_length released_final_region FC_to_Conv_without_reset cores=16
PASS HOST_STREAM accepted_writes=73028 consecutive_writes=72975

[U8/V16: full_model_u8_v16/run.log]
PASS FULL_VGG layers=16 checked_words=22986 class=3 activation_host_words=384
  command_cycles=23539574 preload_cycles=1365024 debug_read_cycles=91937 no_intermediate_reload
```

Fig. 10. RTL verification logs for U8/V8 integration and U8/V16 full-model execution.

5. 결론

본 과제는 지정주제의 Winograd와 행 단위 sparsity-aware 연산을 16코어 SRAM 가속기로 확장하였다. 코어당 16개 MAC lane을 유지하면서 출력 channel을 분할하고, 로컬 U/V 재사용과 A/B activation 역할 전환, multicast 전달 완료를 제어하였다. 이를 통해 코어 간 partial-sum 합산 없이 분산된 출력을 다음 layer의 입력으로 연결하고 Conv와 FC를 실행하는 데이터 경로를 구현하였다.

U8/V16 전체 모델의 선행 실행에서 중간 출력 22,986 words와 최종 분류가 정수 기준 모델에 일치하였다. U8/V8 실제 13개 convolution의 독립 실행에서는 sparsity-aware 처리가 Winograd의 MAC 투입량을 추가 30.76% 줄였으며, dense direct convolution 대비 총 감소율은 69.23%였다. 게이트 수준 통제 실험과 실제 레이어 사례는 이 연산 생략이 공급 조건과 입력 분포에 따라 사이클·에너지 절감으로 연결됨을 보여준다.

전체 16코어와 192개 SRAM을 포함한 합성 면적은 742,748.20 μm^2이며, 합성 후 STA 환산 주파수는 725.89 MHz이다. 고정 셀 매핑 비교에서 sparsity 제어의 추가 면적은 코어당 205.94 μm^2로, SRAM 포함 0.576%, SRAM 제외 로직 기준 1.221%였다. 코어당 ICG 10개의 셀 면적 비중은 0.0073%이다. 이러한 작은 면적 비용으로, zero-row 50% 및 1 word/cycle 공급 조건에서 사이클 39.28%, 에너지 41.64%의 절감을 확인하였다.

다만 연산 생략이 전체 가속기의 처리시간 감소로 이어지는 정도는 데이터 공급에 의해 제한된다. 향후에는 SRAM 접근과 multicast 전송, 외부 가중치 적재의 스케줄을 연산과 더 효율적으로 중첩하고 데이터 재사용을 높여 공급 병목을 줄이는 것이 필요하다.

6. 참고문헌

[1] A. Lavin and S. Gray, "Fast Algorithms for Convolutional Neural Networks," CVPR, pp. 4013–4021, 2016. https://arxiv.org/abs/1509.09308

[2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," ICLR, 2015. https://arxiv.org/abs/1409.1556

[3] NVIDIA, "NVDLA Hardware Architectural Specification." https://nvdla.org/hw/v1/hwarch.html

[4] The OpenROAD Project, "asap7_sram_0p0," BSD-3-Clause. 사용 commit: 9f5af0939e8dd3cc1a9693a50b23441691dd7d25. https://github.com/The-OpenROAD-Project/asap7_sram_0p0

[5] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009. https://www.cs.toronto.edu/~kriz/cifar.html

설계 자료 및 소스: https://github.com/jbpark0616/IDEC_Challenge